

System Design Framework & Fundamentals

① 4-Step Senior System Design Framework (45 mins)



1. Clarify & Scope

- Functional requirements
 - Non-functional (SLA/QPS)
 - Back-of-envelope math
- (5 - 7 mins)



2. High-Level Flow

- API contract (REST/gRPC)
 - Core DB schema entity
 - End-to-end block diagram
- (10 - 15 mins)



3. Deep Dive Core

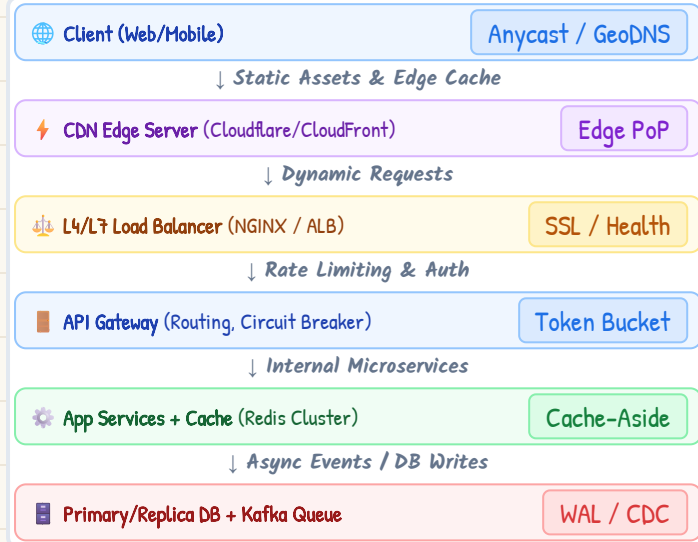
- Scale bottlenecks
 - Cache / Sharding / Queue
 - Algorithms & data models
- (15 - 20 mins)



4. Bottlenecks & Scale

- SPOF & Fault tolerance
 - Monitoring, SLOs, Tracing
 - Cost & trade-offs
- (5 mins)

② End-to-End Architecture Flow



③ Numbers Every 8+ YoE Engineer Must Know

⌚ Latency Hierarchy (Orders of Magnitude)

- L1 cache reference: ~ 0.5 - 1 ns
- L2 cache reference: ~ 3 - 7 ns
- Main Memory (RAM) access: ~ 100 ns
- NVMe SSD random read: ~ 50 - 100 μ s (1000x RAM)
- Read 1 MB sequentially from RAM: ~ 3 μ s
- Read 1 MB sequentially from SSD: ~ 1 ms
- Datacenter network roundtrip (RTT): ~ 0.5 ms
- Intercontinental RTT (US $\leftrightarrow$ EU): ~ 150 ms

📊 Back-of-Envelope Quick Multipliers

- 1 Day = 86,400 s $\approx$ **100K seconds** (for quick mental math)
- 1 Million req/day $\approx$ **12 QPS** | 100M req/day $\approx$ **1.2K QPS**
- Peak Traffic multiplier: Design for **2x to 5x** of average QPS!

④ Availability & SLA Tiers

Availability Tiers (The "Nines")

99.0% ("2 Nines")	3.65 days downtime / year
99.9% ("3 Nines")	8.76 hours downtime / year
99.99% ("4 Nines")	52.6 minutes downtime / year
99.999% ("5 Nines")	5.26 minutes downtime / year (Telco/Fintech)

Key Reliability Metrics

MTBF (Mean Time Between Failures)	System Uptime reliability
MTTR (Mean Time to Repair)	Recovery speed & automated failover
RPO (Recovery Point Objective)	Max acceptable data loss
RT0 (Recovery Time Objective)	Max acceptable downtime

★ Senior Takeaway

✓ Never jump straight to drawing databases or Kafka topics! First establish **Read-to-Write ratio** (e.g., Twitter 100:1 vs Chat 1:1), **storage size per record**, and whether **strong consistency** is non-negotiable (e.g., Payments) or **eventual consistency** is acceptable (e.g., Likes, News Feed).

💡 Staff / Senior Interview Tip

Q: How do you stand out as an 8+ YoE candidate in the first 10 minutes?

A: Don't wait for the interviewer to give you requirements. Actively drive the scope: "Assuming 50M DAU, each user making 20 requests daily yields ~12,000 QPS average, peaking at 35,000 QPS. At 500 bytes per payload, we'll store 500GB/day = 180TB over 5 years. I propose starting with an API Gateway + Cache-Aside architecture to offload 90% of reads."

Scalability, Load Balancing & API Gateways

① Layer 4 (Transport) vs Layer 7 (Application) Load Balancing

Layer 4 Load Balancer (TCP/UDP)

Routing Level: IP + Port (Socket Level)
 Payload Visibility: Blind to HTTP headers/cookies
 Performance: **Ultra-fast, lowest latency (no decrypt)**
 Use Cases: DNS, Gaming, HAProxy TCP mode, AWS NLB

Layer 7 Load Balancer (HTTP/HTTPS)

Routing Level: URL Path, Headers, Cookies, Auth token
 Payload Visibility: Full HTTP message inspection + SSL Termination
 Smart Features: **Path routing (/api/v1/orders), Rate Limit, WAF**
 Use Cases: NGINX, Envoy, Traefik, AWS ALB

VS

② Load Balancing Algorithms & Consistent Hashing



Round Robin / Weighted

Distributes sequentially. Weighted assigns higher traffic to bigger machines.



Least Connections

Routes to instance with fewest active connections. Best for long-lived sessions (WebSockets).



Consistent Hashing

Maps keys & nodes on a 360° ring with virtual nodes. Adding node remaps only K/N keys!



Geo-Proximity

Routes users to the closest datacenter based on GeoDNS latency / BGP Anycast.

③ API Gateway Core Responsibilities & Rate Limiting Algorithms

API Gateway Pattern

- Single Entry Point:** Hides internal microservice topologies.
- Authentication & JWT Validation:** Offloads auth checks.
- SSL / TLS Termination:** Decrypts at perimeter; HTTP internally.
- Protocol Translation:** Translates HTTP/REST to internal gRPC.
- Circuit Breaking:** Fails fast on downstream service degradation.

Rate Limiting Algorithms

1. Token Bucket

Allows Bursts

Tokens added at fixed rate. Request consumes 1 token. If empty, HTTP 429 Too Many Requests.

2. Leaky Bucket

Smooths Out Flow

Requests enter FIFO queue and processed at strictly constant rate. Drops excess requests.

3. Sliding Window Counter

Accurate & Memory-Efficient

Weights previous window + current window: $\text{prev_count} \times (1 - \text{weight}) + \text{curr_count}$.

④ Resilience: Circuit Breaker State Machine

● CLOSED (Normal)

All requests pass through.
 Failures counted over rolling time window.
 If failures > Threshold (e.g. 50%) →

● HALF-OPEN (Testing)

Allows canary sample requests.
 If success → CLOSED
 If failure → OPEN
 ← Timeout expires (e.g. 30s)

● OPEN (Tripped)

Fails immediately (Fast-Fail) or returns fallback cache response. Protects downstream service!

★ Golden Rule for Consistent Hashing

✓ Standard hash (hash(key) % N) causes **99% cache misses** when cluster size \$\$\$ changes! Consistent hashing with **Virtual Nodes (100-300 per physical server)** prevents Hotspots and ensures uniform load distribution.

💡 Staff / Senior Interview Tip

Q: How do you prevent Thundering Herd on cache invalidation or server restart?

A: Use **Mutex / Singleflight** locks (only 1 thread fetches from DB while others wait), **Probabilistic Early Expiration (XFetch algorithm)**, and **Randomized TTL Jitter** so millions of cached keys don't expire simultaneously at the top of the hour.

Distributed Caching & Invalidation Strategies

① 4 Core Caching Patterns (Where & How Writes Occur)



Cache-Aside (Lazy)

- App reads Cache first.
- If Miss → Read DB → Write to Cache.
- ✓ Only caches requested data.
- ✗ Cache misses have 3x latency.



Write-Through

- App writes to Cache.
- Cache synchronously writes to DB.
- ✓ High data consistency.
- ✗ Higher write latency.



Write-Back (Behind)

- App writes to Cache immediately.
- Cache async flushes to DB in batches.
- ✓ Extreme write throughput.
- ✗ Data loss risk if cache dies.



Write-Around

- App writes directly to DB.
- Bypasses Cache completely.
- ✓ Prevents cache pollution.
- ✗ Fresh writes trigger cache miss.

② Cache Production Pitfalls & Battle-Tested Fixes



Cache Avalanche

Problem: Massive number of keys expire at the exact same second, causing DB CPU to spike to 100%.

Solution:

- Add Random TTL Jitter (TTL = base_ttl + rand(1, 300)s).
- Multi-region distributed cache deployment.



Cache Penetration

Problem: Attackers query non-existent keys (e.g. user_id = -999). Misses bypass cache and hit DB directly.

Solution:

- Bloom Filter in front of Cache (O(1) space/time to check existence).
- Cache empty/null results with short TTL (60s).



Cache Breakdown / Stampede

Problem: A single viral "hot key" (e.g. World Cup score) expires; 50,000 concurrent threads hit DB.

Solution:

- Distributed Mutex (Redis SETNX): only 1 worker regenerates cache.
- Background async pre-warm cron job.

③ Redis vs Memcached

Redis

Data Types: Strings, Hashes, Lists, Sets, Sorted Sets (ZSET), Bitmaps, HyperLogLog, Streams
Threading: Single-threaded event loop (I/O multiplexing epoll) + Multi-threaded I/O in v6+
Persistence: RDB snapshots + AOF write log
Replication: Native Master-Replica + Sentinel + Cluster (16,384 Hash Slots)

Memcached

Data Types: Pure Key-Value byte strings only
Threading: True Multi-threaded architecture (scales linearly with CPU cores)
Persistence: None (Pure in-memory, volatile)
Replication: No native clustering (client-side consistent hashing required)

VS

④ Cache Eviction Algorithms



Eviction Policy Cheatsheet

- **LRU (Least Recently Used):** Evicts keys not read for the longest time. (Standard choice; implemented via Doubly Linked List + Hash Map in O(1)).
- **LFU (Least Frequently Used):** Tracks access counters. Evicts least popular items. Great for static catalog access.
- **FIFO:** First-In-First-Out queue order. Simple but suboptimal.
- **2Q / ARC (Adaptive Replacement Cache):** Balances recency and frequency dynamically.



Dual-Write Problem: Invalidate vs Update

- ✓ Never update cache directly on DB write due to race conditions (e.g. Thread A writes DB, Thread B writes DB, Thread B updates Cache, Thread A updates Cache \$to\$ Stale Cache!).
- ✓ Standard Pattern: Write to DB first \$to\$ Delete (Invalidate) from Cache.



Staff / Senior Interview Tip

Q: How do you achieve 100% cache-DB consistency without race conditions?

A: Use Change Data Capture (CDC via Debezium) connected to the Database Write-Ahead Log (WAL). DB commits the transaction \$to\$ Debezium publishes change to Kafka \$to\$ Dedicated consumer service invalidates Redis cache asynchronously with retry guarantees.

Databases: SQL vs NoSQL, Sharding & CAP

① SQL (Relational) vs NoSQL Taxonomy



Relational (SQL)

- Postgres, MySQL
- ACID transactions, strict schema, complex JOINS.
- Storage: B+ Tree indexes.



Document (NoSQL)

- MongoDB, Couchbase
- Flexible JSON/BSON schema, nested sub-objects.
- Great for Catalogs, User profiles.



Wide-Column (NoSQL)

- Cassandra, ScyllaDB, HBase
- High write throughput, LSM-Tree.
- Great for Time-series, IoT, Chat.

Graph DB

- Neo4j, Amazon Neptune
- Nodes, Edges & Properties.
- Social networks, Fraud detection.

② CAP Theorem & The Realistic PACELC Theorem

▲ CAP Theorem (Pick Any 2 during network partition)

- **Consistency (C):** Every read receives the most recent write or an error.
- **Availability (A):** Every non-failing node returns a non-error response (without guarantee of latest data).
- **Partition Tolerance (P):** System continues operating despite network drops between nodes. (Mandatory in distributed systems!).
- **CP Systems:** Google Spanner, HBase, ZooKeeper.
- **AP Systems:** Apache Cassandra, CouchDB, DynamoDB.

⚖️ PACELC Theorem (Extended Real-World Model)

If there is a **Partition (P)**, trade off **Availability (A)** vs **Consistency (C)**;

Else (E) (normal operation), trade off **Latency (L)** vs **Consistency (C)**.

- **PA/EL:** Cassandra, DynamoDB (Low latency favored over strong consistency in normal state).
- **PC/EC:** Google Spanner, CockroachDB (Strong consistency always prioritized).

③ Database Sharding Strategies & Challenges

1. Range-Based Sharding

Sharded by key ranges (e.g. Users A-D on Shard 1, E-H on Shard 2).

✓ Range queries easy.

✗ Hotspot risk (Celebrity 'J' overloads Shard 3).

2. Hash-Based Sharding

Shard = hash(user_id) % N.

✓ Uniform data distribution.

✗ Range queries require scatter-gather across all shards.

3. Directory / Lookup-Based

Lookup service maps partition key to shard ID.

✓ Flexible dynamic re-sharding.

✗ Lookup service is single point of failure / latency hop.

★ Storage Engine Tradeoff: B-Tree vs LSM-Tree

✓ B+ Tree (Postgres/MySQL): In-place updates, faster reads ($O(\log N)$), higher write amplification.

✓ LSM-Tree (Log-Structured Merge-Tree - Cassandra/RocksDB): Sequential append-only writes to MemTable + WAL, background SSTable compaction. Massive write throughput!

💡 Staff / Senior Interview Tip

Q: How do you handle Cross-Shard Transactions without blocking the entire database?

A: Avoid distributed 2-Phase Commit (2PC) in high-throughput systems because coordinator locks cause high tail latency. Instead, co-locate related records using a shared Composite Partition Key (e.g., tenant_id:user_id), or use an asynchronous Saga Orchestration Pattern.

Asynchronous Messaging & Distributed Transactions

① Apache Kafka vs RabbitMQ Deep Comparison

Apache Kafka (Distributed Event Streaming)		RabbitMQ (Traditional Message Broker)	
Model:	Distributed Commit Log (Partitioned append-only file)	Model:	AMQP Broker with Exchanges (Direct, Fanout, Topic)
Consumption:	Pull model (Consumers track own offset pointer)	Consumption:	Push model (Broker pushes to consumers)
Message Retention:	Time/Size based (e.g. 7 days). Replayable!	Message Retention:	Deleted immediately after consumer ACK
Throughput:	Millions msg/sec (Zero-Copy OS page cache)	Throughput:	Tens of thousands msg/sec
Ordering:	Strict ordering guaranteed within a single partition	Features:	Complex routing, priority queues, Dead Letter Exchanges

② Delivery Semantics & Idempotency Pattern

At-Most-Once Commit offset before processing. • Messages never reprocessed. • Data loss if consumer crashes!	At-Least-Once Commit offset after DB processing. • Zero data loss. • Duplicates possible on crash.	Exactly-Once (Effective) At-Least-Once + Idempotency: Unique idempotency_key stored in DB deduplication table.
--	--	--

③ Distributed Transactions: Outbox Pattern & Sagas

Transactional Outbox Pattern Problem: Updating DB and publishing to Kafka is NOT atomic (one can fail). Solution Flow: 1. App saves business entity + message into outbox_table in the same local DB transaction. 2. CDC engine (Debezium) reads DB WAL log and publishes to Kafka. ✓ 100% atomic dual-write guarantee without distributed locks.	Saga Pattern for Distributed Workflows Replaces slow blocking Two-Phase Commit (2PC) across microservices: • Choreography: Services listen to domain events and execute local transactions independently. • Orchestration: Central orchestrator (Temporal / AWS Step Functions) coordinates workflow. • On failure, executes explicit Compensating Transactions (e.g. Refund Payment).
---	--

★ Kafka Partitioning Law ✓ In Kafka, 1 partition can only be read by 1 consumer thread in a consumer group at any time! To scale consumer throughput, you must increase the number of partitions. Setting partition key = user_id guarantees all events for that user arrive in exact chronological order.
--

💡 Staff / Senior Interview Tip Q: How do you prevent Poison Pill messages from blocking an entire Kafka partition? A: Implement a Dead Letter Queue (DLQ) with Exponential Backoff + Jitter. On message deserialization or unrecoverable error, write the payload + stack trace to a order-events-dlq topic after \$\$\$ retry attempts, alert on-call, and commit the partition offset so valid customer messages continue processing.

Storage Systems, CDNs & Media Pipelines

① Storage Types: Block vs File vs Object Storage



Block Storage (EBS, SAN)

- Raw disk blocks without metadata.
- Ultra-low latency IOPS.
- Attached to single instance.

Best for: Databases, OS boot disks.



File Storage (EFS, NFS)

- Hierarchical folder structure.
- Shared across multiple servers.
- POSIX compliant file locks.

Best for: Shared configs, Legacy apps.



Object Storage (S3, GCS)

- Flat namespace with metadata.
- Immutable files accessed via HTTP API.
- 99.999999999% (11 9's) durability.

Best for: Videos, Images, Backups.

② Content Delivery Networks (CDN): Push vs Pull

Pull CDN (Origin Fetch - Default)

Workflow: Edge PoP fetches content from Origin server on first user request (Cache Miss), then caches it for TTL.

Storage Cost: **Minimal** (only popular cached assets stored at edge)

Maintenance: Zero origin management; automatic eviction.

Best For: High-traffic web apps, blogs, YouTube thumbnails.

Push CDN (Proactive Upload)

Workflow: Origin server pushes all new/updated content to edge servers proactively prior to user requests.

Storage Cost: Higher (all assets distributed to all edge locations).

Latency: **0ms origin miss latency for all users.**

Best For: Large software updates (iOS updates, game patches).

VS

③ Large Video / File Upload Pipeline (e.g. YouTube, Google Drive)

1. Client requests upload signature from API

Pre-Signed S3 URL (5-min expiry)

↓ Direct Client-to-S3 bypasses API Servers

2. Client uploads file in SMB chunks in parallel

S3 Multipart Upload / TUS Protocol

↓ S3 Object Created Event Trigger

3. S3 triggers SNS/SQS Transcoding Queue

Event-Driven Pipeline

↓ Distributed Worker Cluster (FFmpeg on GPU)

4. Workers encode video into HLS / DASH (1080p, 720p, 480p)

HLS .m3u8 manifests + .ts segments

★ Never Stream Large Files Through App Servers!

✓ Streaming 2GB video uploads through your API Gateway / Node.js servers exhausts thread pools and network bandwidth immediately! Always generate an Amazon S3 Pre-Signed Upload URL or use the TUS open resumable upload protocol for direct client-to-storage transfers.

💡 Staff / Senior Interview Tip

Q: How do you search across billions of text records in sub-50ms?

A: Use an **Inverted Index** (Elasticsearch / OpenSearch). Unlike relational DBs that scan rows, an inverted index maps every unique word/token to a sorted list of Document IDs (Posting List). Combining multiple terms uses fast bitwise AND / OR operations across posting lists.

Distributed Consensus, Coordination & ID Gen

① Consistency Models Spectrum (Strongest to Weakest)



1. Linearizable (Strict)

- 1. Linearizable (Strict)

All operations appear instantaneous on a global real-time clock. Total global order.



2. Sequential

- 2. Sequential

All operations seen in the same order by all nodes; may lag real physical clock.



3. Read-Your-Own-Writes

- 3. Read-Your-Own-Writes

A user always sees their own updates immediately. Other users see it eventually.



4. Eventual Consistency

- 4. Eventual Consistency

If no new updates, all replicas eventually converge to identical state.

② Consensus (Raft Algorithm)



Raft Core Mechanics (etcd, Consul)

- **Leader Election:** Randomized election timers (150-300ms) prevent split-vote deadlocks.
- **Quorum Rule:** Requires majority vote ($Q = \lfloor N/2 \rfloor + 1$). A 5-node cluster survives 2 node failures.
- **Log Replication:** Leader appends entry → replicates to followers → commits once majority ACKs.
- **Split-Brain Proof:** Only 1 partition can have a majority quorum.

③ Distributed Locks & Fencing Tokens



The Dangerous Distributed Lock Trap

Client 1 acquires lock → JVM triggers Full GC pause for 15s → Lock TTL expires → Client 2 acquires lock → Client 1 wakes up and writes to DB → **Data Corruption!**

Mandatory Fix: Fencing Tokens

Lock service issues monotonically increasing token (e.g. 33, 34, 35). Storage engine rejects writes with token < current highest token.

④ Twitter Snowflake 64-Bit Unique ID Generator



64-bit Binary Bit Layout (Roughly Time-Sorted)

1 Bit

Sign (0)

41 Bits: Timestamp

Epoch ms (69 yrs)

10 Bits: Machine ID

1024 workers

12 Bits: Sequence

4096 IDs/ms/node

- Generates **4,096,000 unique sortable IDs per second per server** without cross-network synchronization!



NTP Clock Drift is Real

- ✓ Physical server clocks drift via NTP adjustments or leap seconds. Never rely on wall-clock time for strict distributed ordering. Use **Logical Clocks (Lamport Timestamps / Vector Clocks)** or hardware atomic clocks with bounded uncertainty intervals (**Google Spanner TrueTime API**).



Staff / Senior Interview Tip

Q: Why is Redis Redlock controversial for distributed locking?

A: Distributed systems expert Martin Kleppmann showed that Redlock relies on synchrony assumptions (bounded network delays and monotonic physical clocks). If a process experiences an unpredicted GC pause or VM migration, Redlock can grant two clients the lock simultaneously. For safety-critical state, always use **etcd / ZooKeeper linearizable sessions with fencing tokens**.

FAANG Blueprints & 8+ YoE Staff Playbook

① 6 Core Product System Blueprints (Cheat Sheet)

1. TinyURL / Bitly

- **Encoding:** Base62 (a-z, A-Z, 0-9). 7 chars = 3.5 Trillion URLs.
- **KGS:** Pre-generate unique keys to avoid DB collision loops.
- **Ratio:** 100:1 read-heavy → 95% served from Redis.

2. WhatsApp / Slack Chat

- **Protocol:** Full-duplex WebSockets.
- **Session:** Presence Servers maintain connection map.
- **Storage:** HBase / Cassandra (LSM-Tree for chat history).

3. Twitter / Instagram Feed

- **Hybrid Fanout:**
 - Regular users: *Fanout-on-Write (Push)*.
 - Celebrities (>100K followers): *Fanout-on-Read (Pull)* merged dynamically at query time.

4. Distributed Rate Limiter

- **Storage:** Redis Cluster.
- **Script:** Atomic **Lua Script** executes Sliding Window Counter in 1 roundtrip.
- **Sync:** Local In-memory cache + async Redis sync.

5. Multi-Channel Notifications

- **Channels:** APNs (iOS), FCM (Android), SMS, Email.
- **Priority Queues:** High (OTP) vs Low (Promo).
- **Deduplication:** 5-min sliding window check.

6. Distributed Web Crawler

- **URL Frontier:** Priority + Politeness FIFO queue.
- **Deduplication:** Bloom Filter for visited URLs.
- **Parser:** Distributed HTML scrapers + S3 storage.

② What Distinguishes an 8+ YoE Senior/Staff Candidate?

🎯 Mid-Level vs Senior / Staff Behaviors

Area	Senior / Staff Engineer Mindset
Ambiguity:	Asks targeted clarifying questions, states assumptions explicitly.
Trade-offs:	Never says "X is best". Explains <i>why</i> X fits this workload despite its downsides.
Failure Modes:	Designs proactively for hardware crashes, split-brains, network partitions.
Operational:	Considers observability (OpenTelemetry), deployment strategy & AWS cloud cost.

📊 Observability Pillar (Metrics, Logs, Traces)

- **Golden Signals:** Latency (p50/p95/p99), Traffic (QPS), Errors (5xx rate), Saturation (CPU/Memory/Disk I/O).
- **Distributed Tracing:** Trace ID & Span ID propagated across all microservices headers (W3C TraceContext / Jaeger).
- **Graceful Degradation:** Serve cached / read-only content if write path is impaired.

★ The Ultimate Interview Checklist

- ✓ **1. Functional:** Top 2-3 core user journeys.
- ✓ **2. Non-Functional:** Scale, Availability (99.99%), Latency (p99 < 100ms), Consistency.
- ✓ **3. Data Model:** SQL vs NoSQL, Primary/Foreign keys, Access patterns.
- ✓ **4. Deep Dive:** Cache invalidation, Kafka partitions, DB sharding, Lock safety.
- ✓ **5. Resiliency:** Circuit breakers, Dead letter queues, Zero Single Points of Failure (SPOF).

💡 Final Staff Interview Wisdom

"System design is not about memorizing architectures; it is the art of balancing trade-offs under constraints." Always drive the conversation, defend your design decisions with data, and guide the interviewer through bottlenecks before they even have to point them out!